W2D1

Created time	October 21, 2026 3:31 PM

Q1.

Source	Type of data	How often it may change	One check before using it
MySQL orders	Structured	Frequently	Need to check row count and required columns
Supplier CSV	Structured text file	When supplier sends a new file	Need to check header and data types
Website JSON	Semi-structured	Frequently	Need to checkexpected fields
Excel stock report	Structured file	Daily or when store sends	Need to check sheet, columns, and missing values

Q2

A. Data engineering
B. Data analytics
C. Data science
D. Data engineering
E. Data engineering

Q3.

```
import pandas as pd

batch_1 = pd.DataFrame({
"order_id": [101, 102],
"city": ["London", "Manchester"],
"amount_gbp": [45.50, 18.00],
})

batch_2 = pd.DataFrame({
"order_id": [103, 104],
"city": ["Bristol", "London"],
```

```
"amount_gbp": [62.25, 27.40],
})

batch_3 = pd.DataFrame({
"order_id": [105, 106],
"city": ["Leeds", "Manchester"],
"amount_gbp": [34.80, 51.10],
})

print("Batch 1:", len(batch_1))
print("Batch 2:", len(batch_2))
print("Batch 3:", len(batch_3))

all_orders = pd.concat(
[batch_1, batch_2, batch_3],
ignore_index=True,
)

print("Total rows:", len(all_orders))
print("Total sales:", all_orders["amount_gbp"].sum())
```

Q4. B Building a reliable process that moves and cleans order data

Q5.

Read new order records
Store the received records
Clean invalid values
Create daily sales summary
Make the summary available to analysts
Check that the run completed

Q6.

Pipeline step	What happens?	Simple check
Source	Supplier sends order file	File exists
Ingestion	Read the file	Expected columns are present
Storage	Keep the received data	Row count matches input
Processing	Clean and transform data	Invalid rows are counted
Serving	Create useful sales output	Business totals are checked

Pipeline step	What happens?	Simple check
Monitoring	Record whether the run succeeded	Status and counts are available

Q7.

```
batch_1["batch_id"] = "batch_1"
batch_2["batch_id"] = "batch_2"
batch_3["batch_id"] = "batch_3"

all_orders = pd.concat(
[batch_1, batch_2, batch_3],
ignore_index=True,
)

summary = (
all_orders
.groupby("batch_id", as_index=False)
.agg(
row_count=("order_id", "count"),
sales_gbp=("amount_gbp", "sum"),
)
)

print(summary)
```

Q8 A. Moving data from a source into the data platform

Q9.

```
import pandas as pd

orders = pd.DataFrame({
"order_id": [201, 202, 203, 204],
"city": [" London ", "Manchester", "Leeds", None],
"amount_gbp": [45.50, -5.00, 30.00, 20.00],
})

valid_mask = orders["city"].notna() & (orders["amount_gbp"] > 0)

valid_orders = orders[valid_mask].copy()
rejected_orders = orders[~valid_mask].copy()

valid_orders["city"] = valid_orders["city"].str.strip()
```

```
print("Valid orders:")
print(valid_orders)

print("\nRejected orders:")
print(rejected_orders)
```

Q10.

Requirement	Choice	Reason
Daily supplier file	Batch	File arrives daily as a complete dataset
Payment events within seconds	Streaming	very less delay is important
Monthly finance report	Batch	Its periodic
Live website activity	Streaming	Events happen in continuous manner

Q11.

```
batch_1 = [10, 20, 30]
batch_2 = [40, 50]
batch_3 = [60, 70, 80, 90]

batches = [batch_1, batch_2, batch_3]

cumulative_total = 0

for batch_number, batch in enumerate(batches, start=1):
record_count = len(batch)
batch_total = sum(batch)
cumulative_total += batch_total

print(f"Batch number: {batch_number}")

print(f"Record count: {record_count}")
print(f"Batch total: {batch_total}")
print(f"Cumulative total: {cumulative_total}")
print("-" * 25)
```

Q12. C. Fraud events that must be checked within seconds

Q13.

A. Record a new customer order. —→ OLTP
B. Update the payment status of one order. →OLTP

C. Calculate sales by city for the last 12 months. ——→ OLAP
D. Compare average order value across product categories. → OLAP

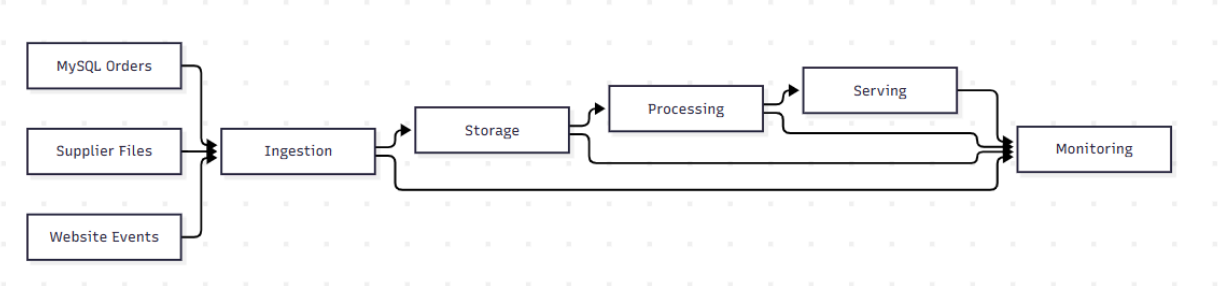
Q14. Design A: ETL

Extract source data
Clean it before loading
Load only cleaned data to the target

Design B: ELT

Extract source data
Load the source data to the target platform
Transform it inside the target

Q15.



Q16. D. Calculate three years of sales by region

Q17.

1. Check the batch status: Confirm that batch_1 was processed successfully, batch_2 failed, and batch_2 has not started. Check the error logs to understand why batch_2failed.

2. Check for partial processing: Verify whether any records from batch_2were processed before the failure. This helps prevent duplicate records when rerunning it.

3. Rerun the failed batch: Fix the cause of the error and rerun batch_2only. I would not rerun batch_1 because it has already succeeded.

4. Verify the results: Confirm that batch_2completed successfully and that its record counts and totals match the expected values.

5. Continue with batch_3 : Once batch_2is successful, start processing batch_3 .

6. Track batch counts: Maintain a log containing the batch number, record count, successful and failed record counts, batch total, processing status, and cumulative total. Update the log after each run and record any retries.

Q18. Design B is generally better for reliable data pipelines. Storing the received data before cleaning creates a recoverable copy of the original input. This is especially useful for daily batch processing, where a failed batch may need to be rerun.

Q19.

```
expected = {
"batch_1": 100,
"batch_2": 120,
"batch_3": 80
}

processed = {
"batch_1": 100,
"batch_2": 118,
"batch_3": 80
}

for batch, expected_count in expected.items():
processed_count = processed[batch]
difference = expected_count - processed_count

 print(f"{batch}: Difference = {difference}")

if difference != 0:
print(f"  Investigation required for {batch}")
```

Investigate Batch_2: Difference is 2, meaning 2 records are missing from the processed count.

Q20.

1. Store sends stock file every evening. —- Batch
2. Website sends click events all day. —— Streaming
3. Finance needs a report at the end of each month. —— Batch

4. Payment team needs alerts within a few seconds. ——Streaming
5. Supplier sends one price file every Monday. —— Batch